

Relatório Técnico — Agente de Revisão de Artigos (baseado em LLM)

Disciplina: Inteligência Artificial — 2026.1 **Questão 3:** aplicação envolvendo **agentes baseados em LLM**. **Sistema:** agente conversacional (LLM via Ollama) que usa um **revisor de artigos baseado em regras** como ferramenta.

1. Visão geral

A questão pede uma aplicação envolvendo **agentes baseados em LLM**. Este projeto implementa um **agente controlado por um LLM** (rodando localmente via Ollama) que avalia a qualidade metodológica de artigos científicos.

O ponto central do desenho é a **divisão de papéis**:

- o **LLM é o agente**: interpreta o pedido do usuário em linguagem natural, **decide**

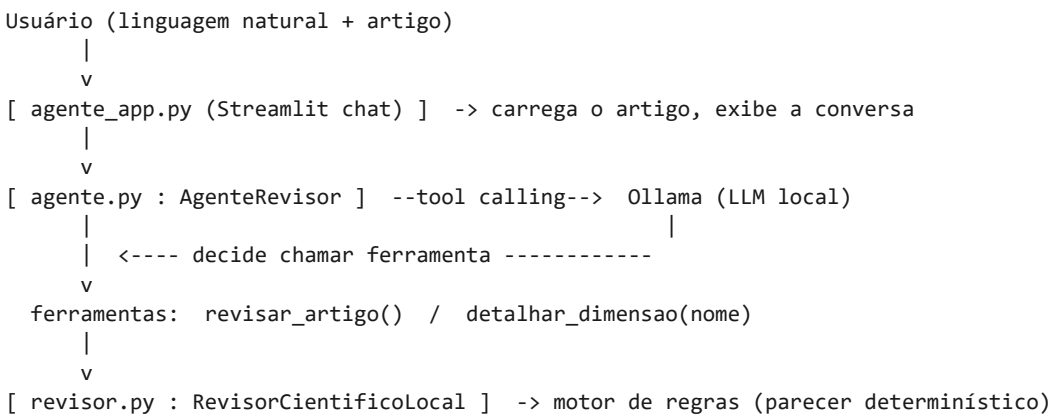
quando chamar as ferramentas disponíveis, integra os resultados e responde de forma conversacional;

- o **raciocínio de conformidade continua simbólico**: quem julga o artigo é um **motor**

de regras determinístico (`revisor.py`), **exposto ao agente como ferramenta**. O LLM não inventa notas nem achados — ele orquestra e explica o parecer real.

Esse desenho é fiel ao princípio pedido também na questão 1 da lista: o LLM apoia a interação, mas **não substitui o mecanismo de raciocínio**.

2. Arquitetura



Componente	Arquivo	Papel
Interface (chat)	agente_app.py	UI Streamlit; carrega o artigo e conduz a conversa.
Agente LLM	agente.py	AgenteRevisor: loop de conversa, <i>tool calling</i> via Ollama, guardrails.

Componente	Arquivo	Papel
Ferramentas	agente.py	revisar_artigo, detalhar_dimensao (fachada sobre o motor de regras).
Motor de regras (a ferramenta)	revisor.py	RevisorCientificoLocal: avalia o artigo por regras de produção.
Interface do revisor (avulsa)	app.py	UI Streamlit que usa o motor de regras diretamente, sem o agente.

3. O agente baseado em LLM

Implementado em agente.py (classe AgenteRevisor). Características:

- **Backend local (Ollama).** A comunicação usa apenas a biblioteca padrão (urllib),

sem dependências pip novas. O modelo padrão é llama3.2 (configurável por OLLAMA_MODEL); ele precisa **suportar tool calling** (function calling).

- **Loop do agente.** A cada mensagem do usuário, o agente envia ao LLM o histórico

mais o esquema das ferramentas. Se a resposta do modelo contém tool_calls, o agente **executa a ferramenta localmente**, devolve o resultado ao modelo e repete; quando o modelo responde sem pedir ferramenta, esse texto é a resposta final. Há um teto de iterações para evitar laços.

- **Guardrails (system prompt).** O agente é instruído a **sempre** chamar

revisar_artigo antes de comentar a qualidade do artigo e a **nunca inventar** notas, achados ou trechos — usando somente o que as ferramentas retornam.

- **Degradação clara.** Diferentemente da questão 1 (onde o LLM é opcional), aqui ele é

essencial. Se o Ollama não estiver acessível, o agente levanta OllamaIndisponivel com instruções, e a interface orienta o usuário a instalar/baixar o modelo.

4. Ferramentas expostas ao agente

O artigo em análise fica no contexto do agente; as ferramentas operam sobre ele:

- revisar_artigo() — executa o motor de regras sobre o artigo carregado e devolve

um parecer compacto: pontuação (0–100), decisão sugerida, nota e nº de achados por dimensão, achados graves, contagem de citações/palavras e seções detectadas.

- detalhar_dimensao(dimensao) — recorta os achados detalhados de uma dimensão

específica do último parecer (ex.: "metodologia", "ética"), para perguntas de acompanhamento.

Cada ferramenta é declarada em um **esquema JSON** (formato de tools do Ollama) com nome, descrição e parâmetros, para que o LLM saiba quando e como chamá-la.

5. A ferramenta de raciocínio: motor de regras

Quem efetivamente avalia o artigo é o `RevisorCientificoLocal` — um sistema especialista **baseado em regras**, local e determinístico. Resumo do funcionamento:

- **Encadeamento para frente:** normaliza o texto (sem acento, junta hifenização de

quebra de linha, colapsa espaços), segmenta em sentenças, extrai fatos (seções presentes, citações, nº de palavras) e dispara as regras de cada dimensão, acumulando achados e penalidades.

- **Oito dimensões com pesos:** Estrutura (1.0), Problema/objetivo (1.1), Rigor

metodológico (1.4), Evidências (1.3), Referencial (1.2), Argumentação (1.0), Ética e integridade (1.5), Limitações (0.8).

- **Score com gate de gravidade:** média ponderada limitada por um teto — grave de

integridade é eliminatório (teto 25); demais graves reduzem o teto ($\max(40, 100 - 12 \times \text{n}^\circ_{\text{graves}})$); sem graves, a média vale integralmente. Isso impede que um artigo bem formatado, porém fraudulento, receba nota confortável.

- **Detecções de destaque:** fraude/pseudociência (eliminatória), resultado numérico

não rastreável, amostra pequena não justificada, fonte fraca como base, linguagem absoluta e subjetividade usada como critério.

*O motor de regras é o mesmo descrito em detalhe no histórico do projeto; aqui ele passa a ser a **ferramenta** consultada pelo agente.*

6. Interface com o usuário

- `agente_app.py` (foco da questão): um **chat** em Streamlit. O usuário carrega o

artigo (texto ou PDF) na barra lateral e conversa com o agente; a interface mostra **quais ferramentas foram chamadas** em cada resposta, evidenciando o comportamento de agente. Se o Ollama não estiver disponível, exibe as instruções de configuração.

- `app.py` (avulso): a interface anterior, que usa o motor de regras diretamente

(sem LLM), continua disponível para inspecionar o parecer completo.

7. Testes automatizados

`testes/test_agente.py` (pytest) trava o **contrato entre o agente e o motor**: a tool `revisar_artigo` reflete o parecer real (fraude é eliminatória, ≤ 25 e "Não recomendado na forma atual"), `detalhar_dimensao` recorta a dimensão pedida e trata dimensão inexistente, ferramenta desconhecida e artigo vazio retornam erro controlado. O loop de conversa com o LLM depende do servidor Ollama e é validado na demonstração. `testes/test_revisor.py` mantém os testes do

motor de regras.

8. Limitações e possíveis melhorias

Limitações. (a) O agente depende de um Ollama local com modelo que suporte *tool calling*; a qualidade das respostas varia com o modelo. (b) O julgamento herda as limitações do motor de regras (análise léxica, não semântica). (c) PDFs escaneados não têm texto extraível.

Melhorias futuras. (a) Mais ferramentas (ex.: comparar dois artigos, sugerir reescrita de um trecho); (b) memória de conversas entre sessões; (c) suporte a modelos via API na nuvem como alternativa ao Ollama; (d) *streaming* das respostas na interface.

9. Como executar

Requer Python 3.11+ e **Ollama** para o agente.

```
# dependências (a partir desta pasta)
python -m pip install -r requirements.txt

# pré-requisito do agente: Ollama com um modelo que suporte tools
ollama pull llama3.2          # com o Ollama instalado e aberto

# agente conversacional (LLM + ferramentas) – foco da questão 3
python -m streamlit run agente_app.py

# revisor de regras avulso (sem LLM)
python -m streamlit run app.py

# testes
python -m pytest
```

10. Mapa dos entregáveis

Entregável	Onde está
Agente baseado em LLM	agente.py + agente_app.py
Ferramenta de raciocínio (motor de regras)	revisor.py
Interface do revisor avulso	app.py
Testes	testes/test_agente.py, testes/test_revisor.py
PDFs de exemplo	testes/artigo teste ok.pdf, testes/artigo_com_falhas.pdf
Relatório técnico	este documento (RELATORIO_TECNICO.md / .pdf)